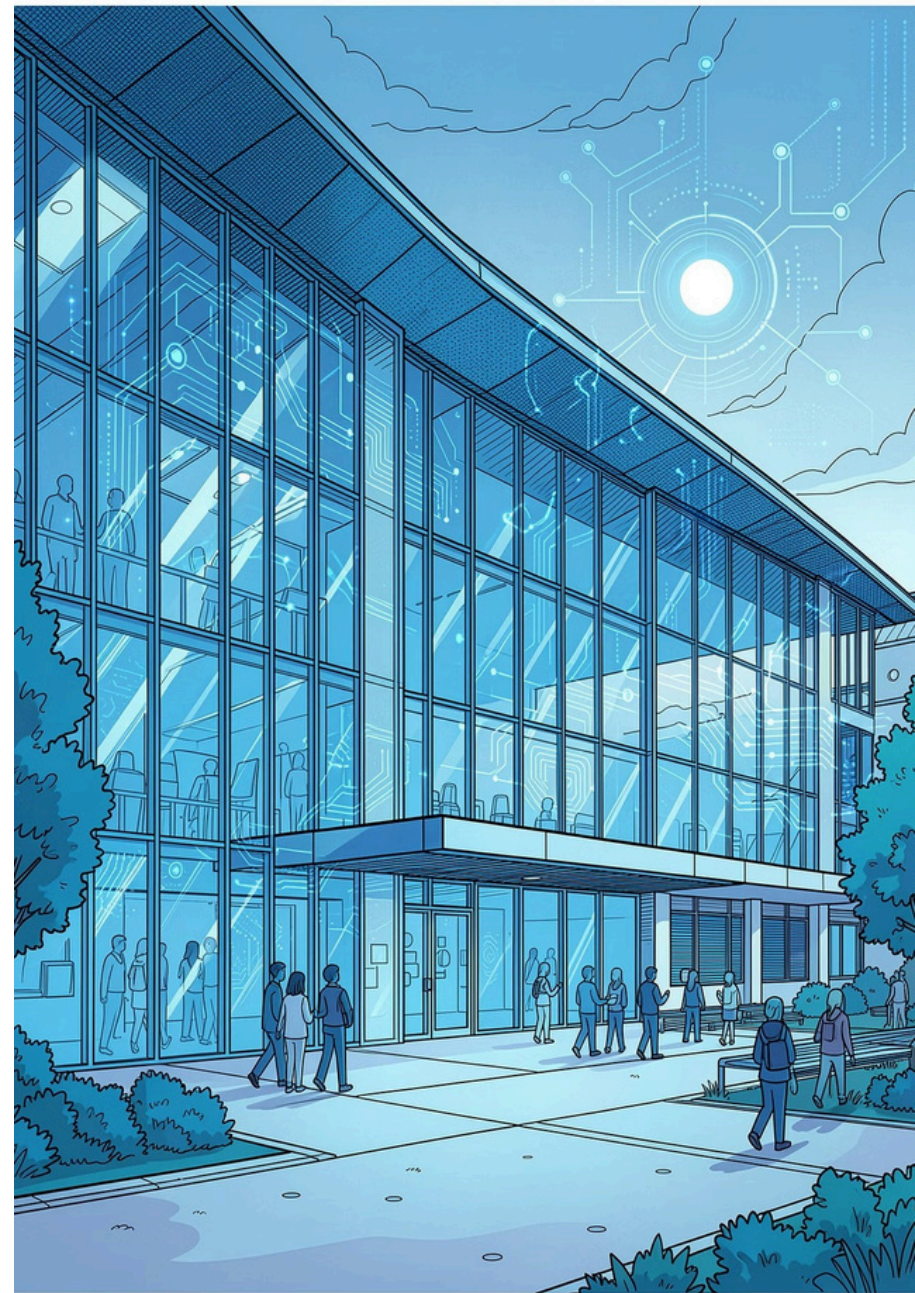


Sistema de Gestión Escolar Colegio Bernardo O'Higgins

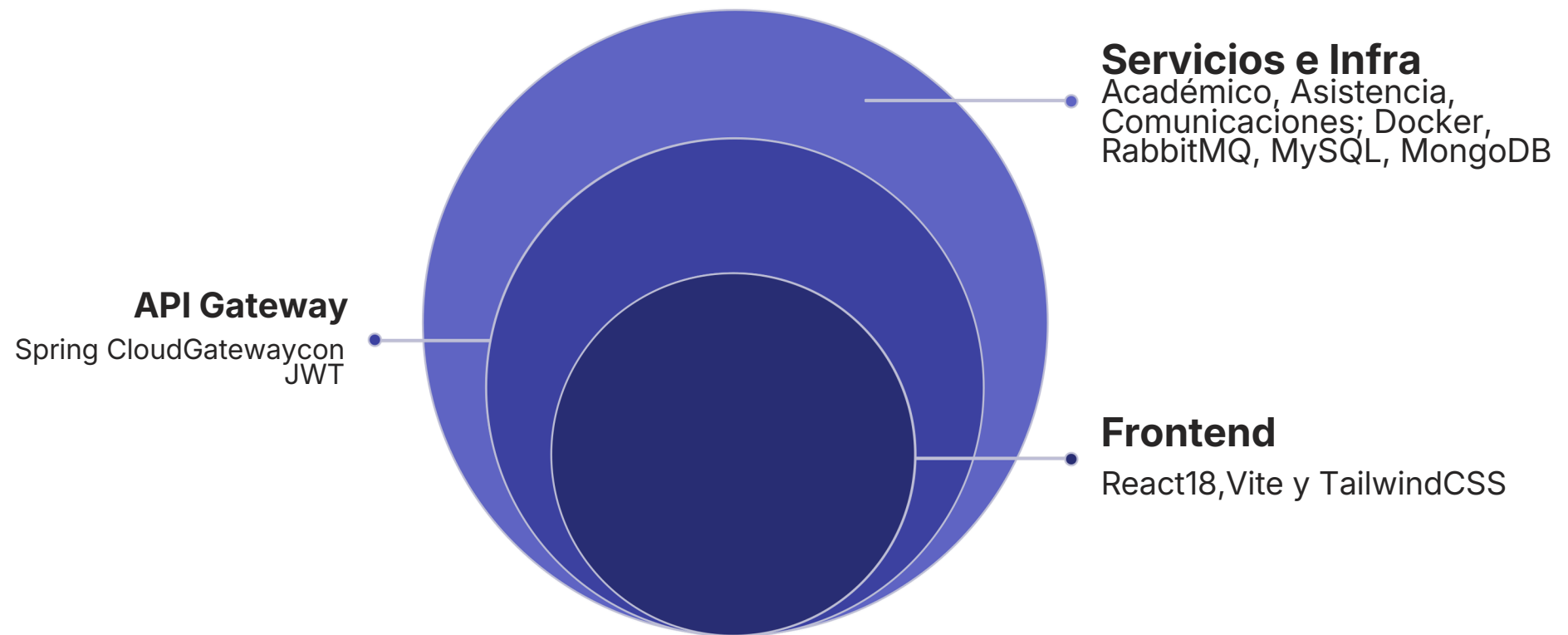
DEFENSA DE PROYECTO

SPRINGBOOT · REACT · DOCKER



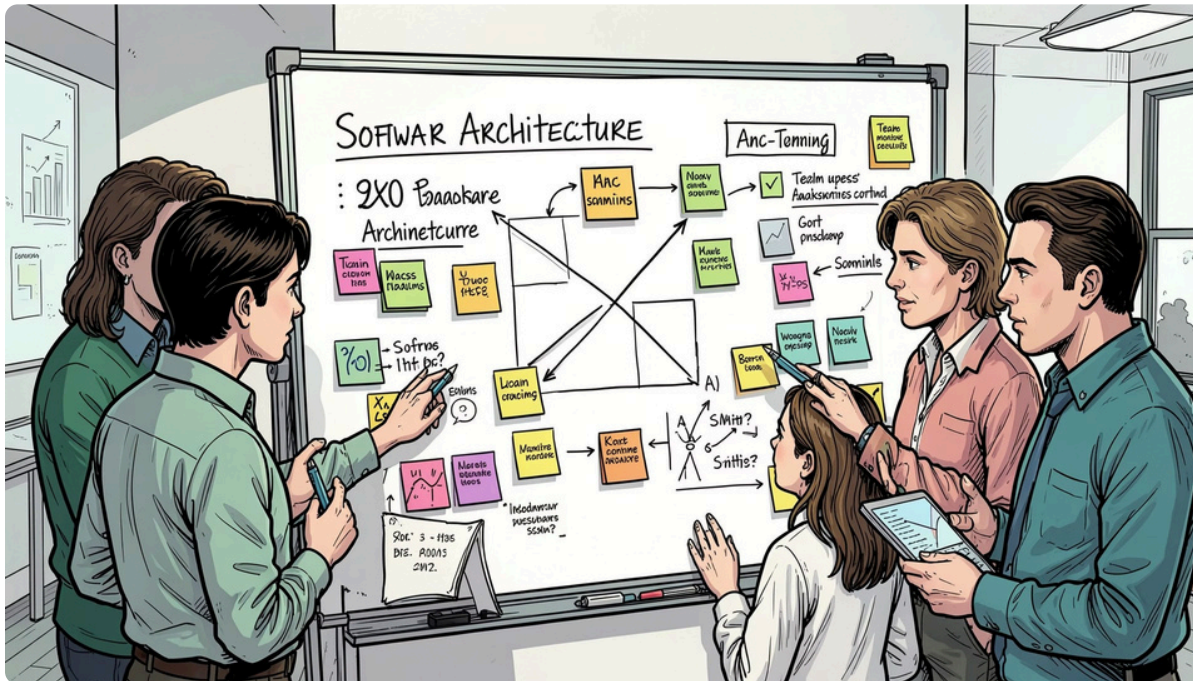
Arquitectura del Sistema

El sistema se estructura en **5 microservicios independientes** orquestados mediante Docker Compose, con comunicación síncrona y asíncrona según la criticidad de cada operación.



Cada capa tiene responsabilidad acotada, lo que permite escalar, desplegar y mantener los servicios de forma independiente.

Técnicas de Ideación y Justificación de Microservicios



Técnicas aplicadas

- **EventStorming:** identificación de dominios y eventos de negocio
- **Domain-Driven Design:** límites contextuales por subdominio
- **Análisis de cohesión y acoplamiento:** criterios de separación

¿Por qué microservicios?

El colegio requiere módulos con ciclos de vida distintos: notas, asistencia y comunicaciones evolucionan de forma independiente. La separación garantiza **despliegues autónomos**, tolerancia a fallos y escalado granular.

Stack Tecnológico



Frontend

React18+Vite + TailwindCSS + Axios + Context API



Microservicios

SpringBoot+JPA+MySQL 8.0 · OpenFeign · MongoDB 6.0



Pruebas

JUnit5+Mockito + MockMvc + JaCoCo + SonarQube



API Gateway

SpringCloudGateway + Spring Security + JWT (jjwt 0.12.6)



Mensajería

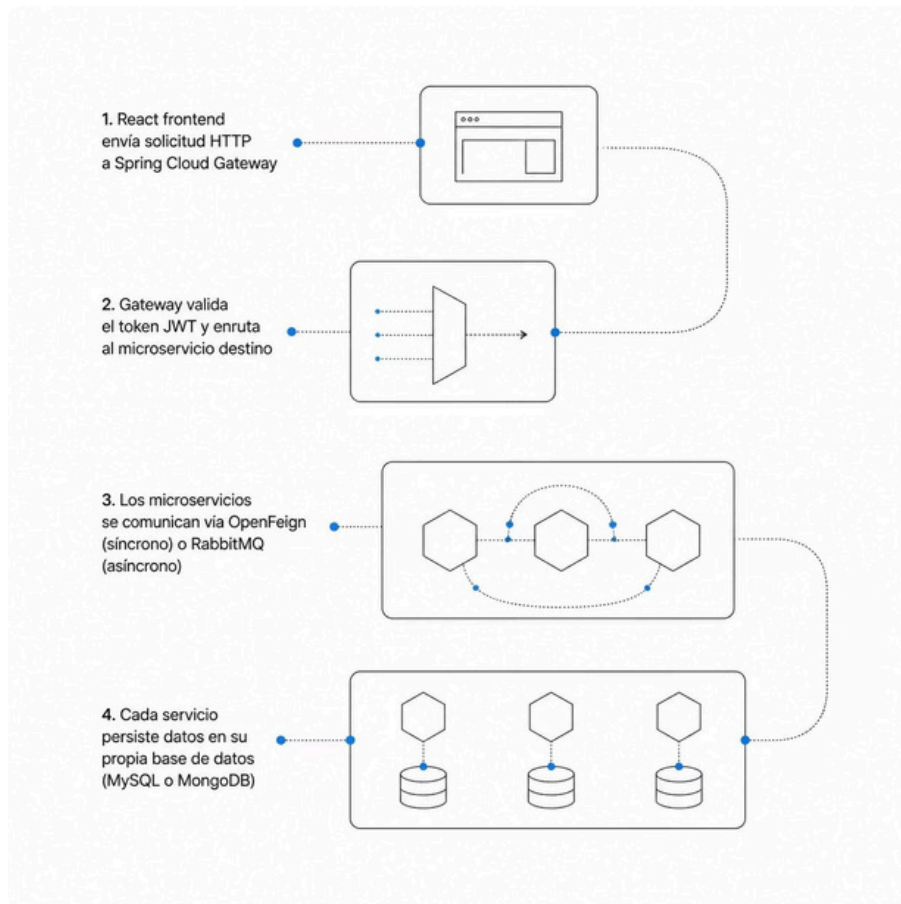
RabbitMQparacomunicación asíncrona entre Asistencia y Comunic aciones



Infraestructura

DockerComposecon 10 contenedores orquestados

¿Cómo se integran las tecnologías?



Patrón de integración

El **APIGateway** centraliza autenticación y enrutamiento. Los servicios se comunican mediante **OpenFeign** para llamadas síncronas y **RabbitMQ** para eventos asíncronos, evitando acoplamiento directo.



La base de datos por servicio (MySQL o MongoDB) garantiza independencia de persistencia y evita el acoplamiento en el esquema de datos.

Integración Frontend / Backend



Frontend → Backend

- **Axios** gestiona todas las peticiones HTTP con interceptores para el token JWT
- **Context API** centraliza el estado de autenticación y datos del usuario
- El Gateway devuelve **401/403** ante tokens inválidos; el frontend redirige al login

Escalabilidad demostrada

Cada microservicio puede escalarse horizontalmente sin afectar al resto. El Gateway distribuye la carga y los servicios se descubren dinámicamente en tiempo de ejecución.

Estrategia de Pruebas

Se aplica la **pirámide de pruebas** con foco en unitarias e integración, asegurando cobertura real medida con JaCoCo y validada en SonarQube.

72.7%

Filtros de seguridad y enrutamiento

94.1%

Eventos y colas RabbitMQ

92.6%

Lógica de negocio crítica

94.9%

Consumer MongoDB + RabbitMQ

Patrones de Diseño y Mantenibilidad

Patrones aplicados

- **API Gateway:** punto único de entrada y seguridad centralizada
- **Circuit Breaker (Resilience4j):** tolerancia a fallos en llamadas entre servicios
- **Publicador/Suscriptor:** RabbitMQ desacopla Asistencia de Comunicaciones
- **Repository + Service:** separación de capas en cada microservicio

Impacto en la calidad

Estos patrones permiten modificar un servicio sin afectar a los demás, facilitan la sustitución de componentes y reducen el riesgo en cada despliegue. SonarQube valida además la deuda técnica y las vulnerabilidades de seguridad.

 La cobertura media supera el **88%**, muy por encima del umbral habitual del 80% en proyectos académicos.

Infraestructura Docker Compose

El sistema completo se despliega con **10 contenedores** orquestados mediante Docker Compose, garantizando reproducibilidad y aislamiento del entorno.

Frontend

React 18 + Vite

Gateway

Spring Cloud Gateway

Académico

Spring Boot + MySQL

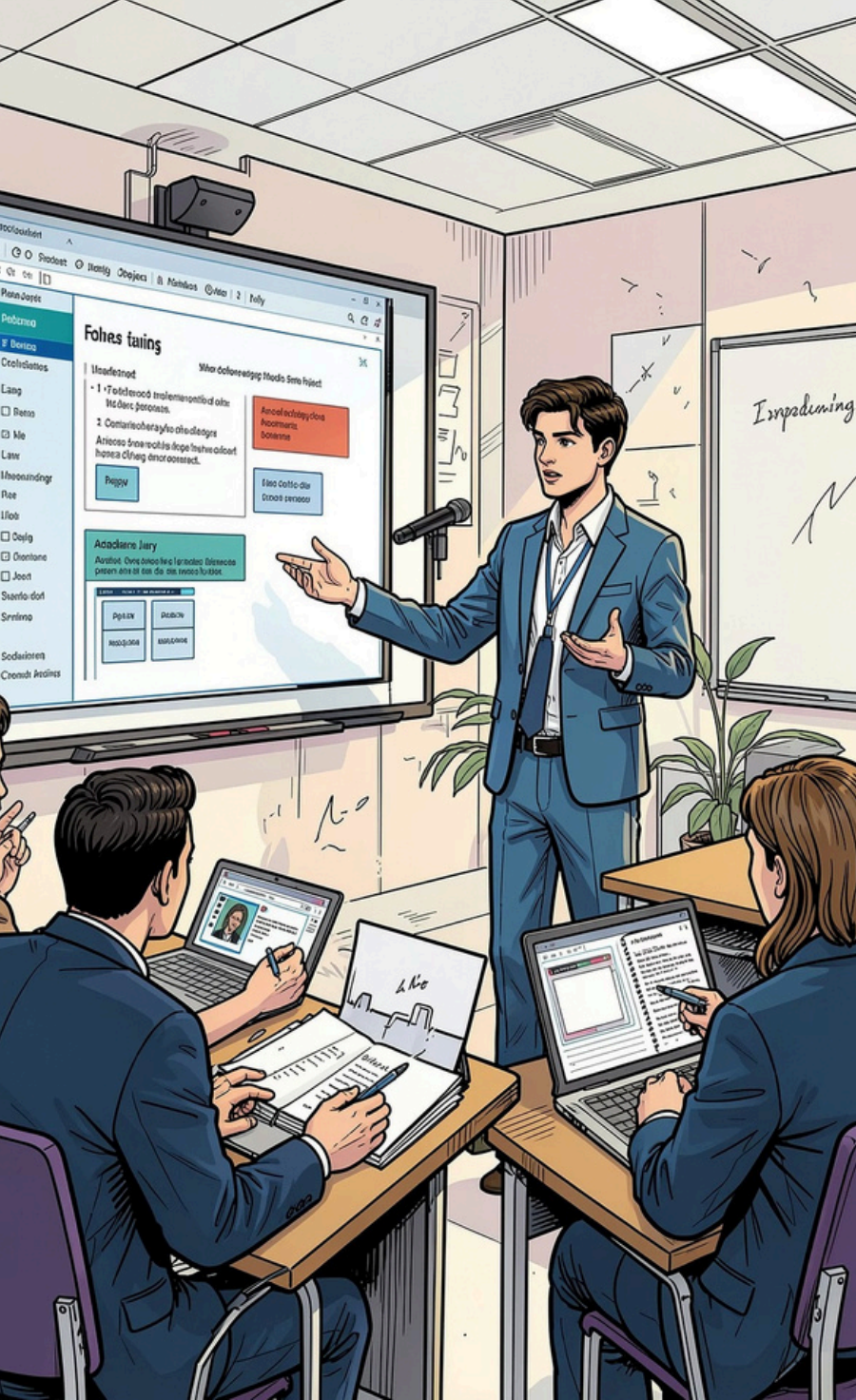
Asistencia

Spring Boot + RabbitMQ

Comunicaciones

Spring Boot + MongoDB

Los contenedores restantes corresponden a las bases de datos (MySQL x2, MongoDB), RabbitMQ y los servicios de soporte necesarios para el funcionamiento completo del ecosistema.



Cierre y Próximos Pasos

Lo que se ha demostrado

Arquitectura de microservicios funcional, con integración frontend/backend, pruebas con cobertura superior al 88% y despliegue reproducible en Docker.

Evolución futura

Se contempla la incorporación de Kubernetes para orquestación en producción, monitorización con Prometheus/Grafana y la extensión a nuevos dominios como gestión de pagos y biblioteca.

El sistema no solo cumple los requerimientos del cliente, sino que establece una base sólida y escalable para el crecimiento futuro del colegio.